SAVEETHA SCHOOL OF ENGINEERING
C PROGRAMMING ASSIGNMENT – Grama Jyothi Billing Program

Field	Entry
Student Name	BAVADARANI S
Register Number	1912260562
Course Code & Name	CSA0209 C PROGRAMMING
Slot	B
Assignment / Task Title	Design, Implement and Validate a Reliable Billing Program
GitHub Repository URL	https://github.com/bavadarini28 /CSA0209-1912260562-BAVADARANI.S.git
Submission Date	12 NOVEMBER 2026

TASK 2 – Choose the Data Types

Aim

To choose a suitable C type for each data item, using sizeof and the limits of the types on the system.

Data-type table (measured on my system)

Data item	Type	sizeof	Range / reason
Meter reading	unsigned short (meter_t)	2	0 – 65535; matches the 16-bit register
Household number	int (or unsigned int)	4	up to 2,147,483,647 ≥ 800,000
Units consumed	unsigned int	4	0 – 4,294,967,295; never negative
Category	char	1	small code
One bill (paise)	long	8*	±9.22×10^18 on Linux/64-bit (*4 bytes on Windows!)
Total for district	long long	8	±9.22×10^18 guaranteed at least 64-bit
Rates, slab limits	int / long (macros)	4 / 8	whole paise / whole units

Measured: char 1, short 2, int 4, long 8, long long 8, float 4, double 8. INT_MAX = 2147483647, LLONG_MAX = 9223372036854775807, USHRT_MAX = 65535, FLT_DIG = 6, DBL_DIG = 15. A district total of about 22,040,000,000 paise exceeds INT_MAX, so long long is the safe choice.

Why not float rupees? Values such as 2.25 are exact in binary, but 162.60 or 0.10 are not; float has only about 6–7 significant digits, so errors accumulate. Storing paise in an integer is exact because every amount is a whole number of paise.

Rollover: after 65535 an unsigned short wraps to 0 (arithmetic is modulo 65536).

Precision demonstration

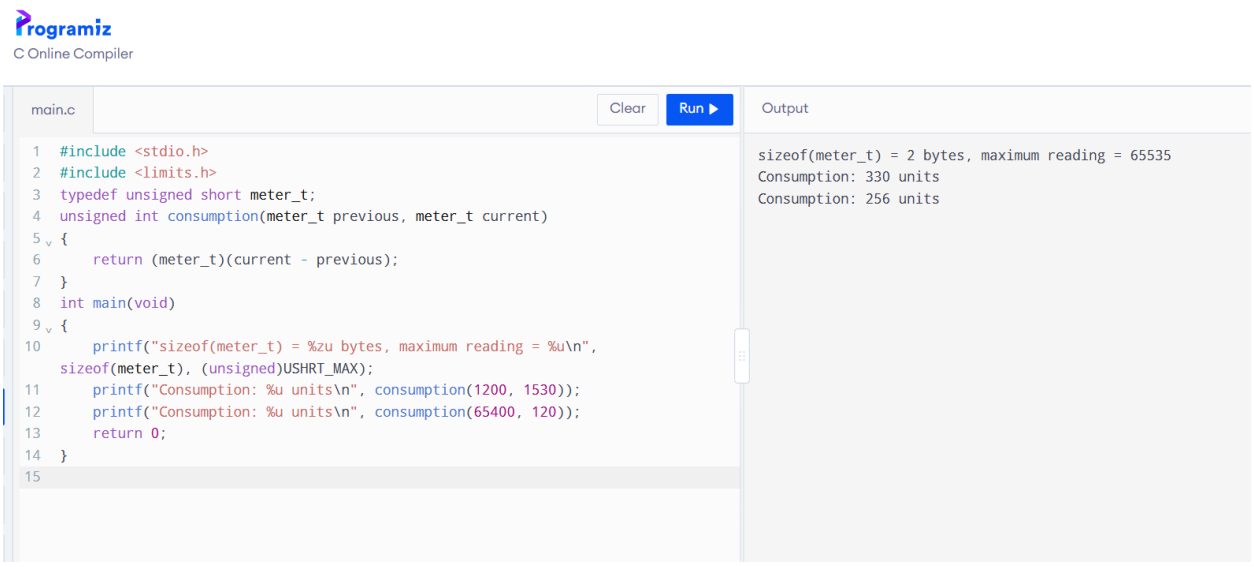
```
float f=0; long p=0;
for (int i=0;i<4000;i++){ f+=162.60f; p+=16260; }
/* float  = 650405.1875   (wrong by Rs 5.19)
   paise  = Rs 650400.00  (exact)          */
```

Complete the Code – TODO 1

```
#include <stdio.h>
#include <limits.h>
typedef unsigned short meter_t;
unsigned int consumption(meter_t previous, meter_t current)
{
   return (meter_t)(current - previous);
}
int main(void)
{
   printf("sizeof(meter_t) = %zu bytes, maximum reading = %u\n", sizeof(meter_t), (unsigned)USHRT_MAX);
   printf("Consumption: %u units\n", consumption(1200, 1530));
   printf("Consumption: %u units\n", consumption(65400, 120));
   return 0;
}
```

Output:

```
sizeof(meter_t) = 2 bytes, maximum reading = 65535
Consumption: 330 units
Consumption: 256 units
```



Why 256: 120 − 65400 is promoted to int and equals −65280, but converting it to meter_t (unsigned short) reduces it modulo 65536: −65280 + 65536 = 256. This is exactly the true consumption (65400→65535 is 135 units, 65535→0 is 1 unit, 0→120 is 120 units; 135 + 1 + 120 = 256). No if–else is needed.

Alternative Concept – money as double rupees

Bill = 50.0 + slab charges computed with 2.25 and 4.50, accumulated in a double and rounded with round(x*100)/100 once per bill (and again for totals). Time per addition is O(1) in both; space is 8 bytes for double versus 8 bytes for long – the same on a 64-bit system. The difference is error: integer paise has zero error for any n, whereas double has error that grows roughly like n × 2^-53 × total in the worst case (about √n in the typical case). Double is accurate enough to display after rounding, but equality checks and audits can fail (0.1 + 0.2 ≠ 0.3).

Result: Types justified; precision problem shown (Rs 5.19 error with float) and removed by integer paise.